Apache Kafka → Eventos que conectan servicio

Bus de eventos que permite que los microservicios se comuniquen de forma **desacoplada, asíncrona** y escalable.

Concepto	Qué significa	En TPI
Emisor	El microservicio que genera un hecho/evento y quiere comunicarlo	challenges-service
Producer	El componente/código que publica ese evento en Kafka	Código Kafka Producer dentro de challenges-service
Kafka	El intermediario que recibe, almacena y distribuye los eventos	Kafka / topic challenges.results
Receptor	El microservicio que recibe un evento que le interesa	notifications-service o ranking-service
Consumer	El componente/código que escucha y procesa los mensajes de Kafka	ChallengeEventConsumer

1. Contrato del evento

```
{
  "eventId": "123e4567-e89b-12d3-a456-426614174000",
  "eventType": "EVENT_RESULT",
  "timestamp": "2026-09-19T23:53:00Z",
  "producer": "tema-XX-service-name",
  "payload": {}
}
```

Field	Obligatorio	Función
`eventId`	Sí	Identifica unívocamente el evento
`eventType`	Sí	Indica qué ocurrió
`timestamp`	Sí	Momento en que ocurrió
`producer`	Sí	Microservicio que produjo el evento
`payload`	Sí	Datos específicos del evento

Nota: Todo lo emitido se publica en inglés y bajo este contrato.

2. Tópicos

- Product Domains
- Notifications and LLMs
- Marketplace
- Sandbox
- Security and Audit

Nota: No se le permite a los grupos crear nuevos tópicos → Si necesitan uno, avisan al grupo notificaciones

3. Ejemplo de EMISOR

Supongamos que el grupo de Challenges tiene: **challenges-service → Cuando un alumno termina un desafío, ese microservicio pública:**

- Topic challenges.results
- Event CHALLENGE_COMPLETED

3.1 Dependencia, ambos microservicios necesitan **Spring for Apache Kafka**.

pom.xml

```
<dependency>
   <groupId>org.springframework.kafka</groupId>
   <artifactId>spring-kafka</artifactId>
</dependency>
```

3.2 Contrato común del evento

```
package com.example.events;
import java.time.Instant;
public class Event<T> {

   private String eventId;
   private String eventType;
   private Instant timestamp;
   private String producer;
   private T payload;

   public Event() {
   }

   public Event(
         String eventId,
         String eventType,
         Instant timestamp,
         String producer,
         T payload
   ) {
      this.eventId = eventId;
      this.eventType = eventType;
      this.timestamp = timestamp;
      this.producer = producer;
      this.payload = payload;
   }

   // getters y setters
}
```

Representa:

```
{
  "eventId": "...",
  "eventType": "CHALLENGE_COMPLETED",
  "timestamp": "2026-09-19T23:53:00Z",
  "producer": "challenges-service",
  "payload": {}
}
```

Application.yml

```
spring:
  kafka:
    bootstrap-servers: ${KAFKA_BOOTSTRAP:event-bus:29092}

    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
```

Así el micro sabe dónde está Kafka → event-bus:29092

El evento CHALLENGE_COMPLETED necesita sus propios datos:

```
package com.example.events;

public class ChallengeCompletedPayload {

    private String studentId;
    private String challengeId;

    public ChallengeCompletedPayload() {
    }

    public ChallengeCompletedPayload(
            String studentId,
            String challengeId
    ) {
        this.studentId = studentId;
        this.challengeId = challengeId;
    }

    // getters y setters
}
```

Por lo tanto, nuestro evento completo será: Event<ChallengeCompletedPayload>

4. Ejemplo de **RECEPTOR**

Application.yml

```
spring:
  kafka:
    bootstrap-servers: ${KAFKA_BOOTSTRAP:event-bus:29092}

    consumer:
      group-id: notifications-service
      auto-offset-reset: earliest

      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer

      properties:
        spring.json.trusted.packages: "com.example.*"
```

Así el micro identifica el grupo → group-id: notifications-service

---

```
package com.example.kafka;

import …

@Service
public class ChallengeEventConsumer {

    @KafkaListener(
        topics = "challenges.results",
        groupId = "notifications-service"
    )
    public void consume(Event<?> event) {

        System.out.println(
            "Received event: " + event.getEventType()
        );

        if ("CHALLENGE_COMPLETED".equals(event.getEventType())) {

            System.out.println(
                "Challenge completed!"
            );

            // Procesar evento
            // Crear notificación
        }
    }

}
```

IMPORTANTE

Si challenges publicara:

```
{
  "eventId": "123",
  "eventType": "CHALLENGE_COMPLETED",
  "timestamp": "2026-09-20T14:30:00Z",
  "producer": "challenges-service",
  "payload": {
    "studentId": "std-101",
    "challengeId": "ch-50",
    "score": 95,
    "attemptNumber": 2,
    "language": "JAVA"
  }
}
```

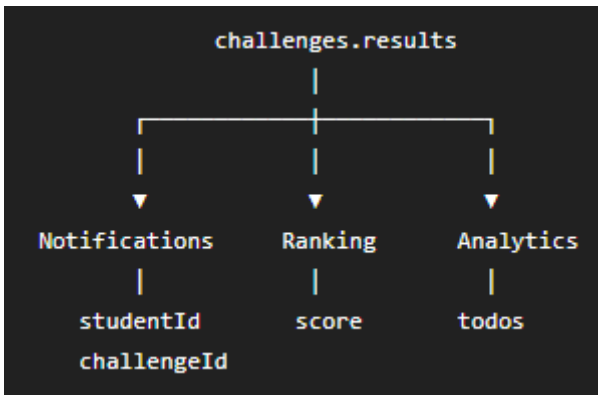
Notificaciones:

```
String studentId =
        event.getPayload().getStudentId();

String challengeId =
        event.getPayload().getChallengeId();
```

Ranking:

```
Integer score = event.getPayload().getScore();
```



Nota: No se debe hacer un evento por consumidor si representan el **mismo hecho de negocio**. Ni tampoco "Metamos todos los datos posibles en CHALLENGE_COMPLETED por si algún microservicio los necesita."

El evento debe contener la información necesaria para describir el hecho ocurrido y permitir que sus consumidores reaccionen, pero no convertirse en un contenedor de todos los datos.

Consumer TIPADO (el tipo de consumer depende de como se define el playload)

Notificaciones:

```
@KafkaListener(
    topics = "challenges.results",
    groupId = "notifications-service"
)
public void consume(
        Event<ChallengeCompletedPayload> event) {

    String studentId =
            event.getPayload().getStudentId();

    String challengeId =
            event.getPayload().getChallengeId();

    // Crear notificación
}
```

Ranking:

```
@KafkaListener(
    topics = "challenges.results",
    groupId = "ranking-service"
)
public void consume(
        Event<ChallengeCompletedPayload> event) {

    Integer score =
            event.getPayload().getScore();

    String studentId =
            event.getPayload().getStudentId();

    // Actualizar ranking
}
```
